Final Project

David Koyrakh

Bellevue University

DSC-650: Big Data

Professor Nasheb Ismaily

November 19, 2025

Final Project

Introduction

For my final project, I customized and deployed a big data pipeline for moving an advertisement-performance dataset through NiFi, HDFS, Hive, Spark (via YARN), and HBase. The pipeline includes building and feeding the data through a Linear Regression model in order to understand the predictive power of certain advertising metrics against total revenue. This exercise demonstrates how the technologies work together in a realistic big data pipeline.

Dataset

I generated a synthetic dataset which contains metrics for 100 online advertising campaigns. It is hosted publicly in my GitHub repo here: https://github.com/dovkoy/big-data-650/blob/trunk/online_ads.csv

The dataset includes eight columns that measure different aspects of an advertisement's performance, such as ad spend, impressions, clicks, conversions, click-through rate (CTR), and revenue. I intentionally kept the dataset small (100 rows) to avoid overloading the VM's limited resources. The file required no complex cleaning, besides removing the header row that Hive initially ingested as a null record. Using this data allowed me to practice orchestrating the pipeline on a real-life business problem: how might online advertisement spending and performance metrics impact a company's total revenue?

Pipeline Overview

This pipeline begins with a NiFi flow for downloading the dataset from my public GitHub repo and writing it to HDFS. The data is then loaded into a Hive table for structured access. Next, Spark (running on YARN) reads the data from Hive, builds and evaluates a linear regression model on it, and outputs metrics such as RMSE and R^2. These model results are then written to an HBase table using HappyBase. This completes the workflow of ingestion, storage, processing, and NoSQL persistence.

Issues Encountered

- My CSV file contained a header row and when initially loading the dataset, I did not include the parameter for skipping the header row. Thus, I had to manually remove the first "NULL" row that was created in the table.
- The master container did not have git or apt-get installed, so I had to clone the repo in the host VM and copy my Spark ML script into the container manually. The command I used to copy from the host to the Docker container was:
    - docker cp /home/dovid/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase/big-data-650/sparkml.py hadoop-hive-spark-hbase_master_1:/root/

Screenshots

I started by loading my dataset into HDFS using the provided template. The template includes a processor for downloading a file from the public internet, passing it through a queue to another processor which uploads it with the correct filename and path, and then through another queue to the final processor, which writes the file to HDFS:

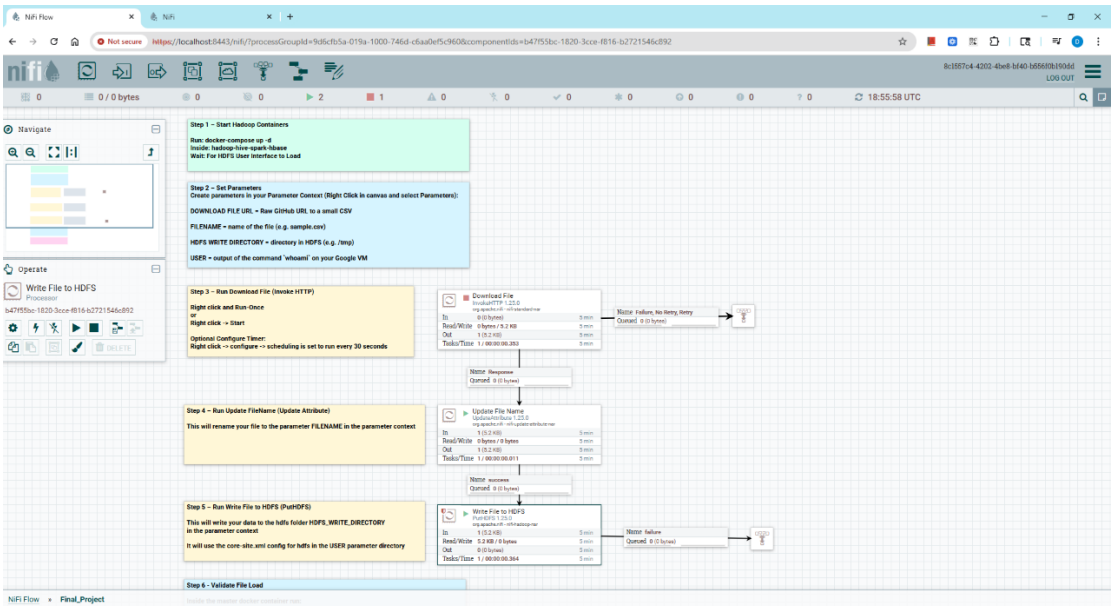

Figure 1: Provided NiFi template running to download my dataset and store it in HDFS.

I then opened the Docker container's master bash shell and verified the presence of my dataset in HDFS. The file correctly showed in the "tmp" directory of HDFS, and running "cat" on the file confirmed its contents are correct:

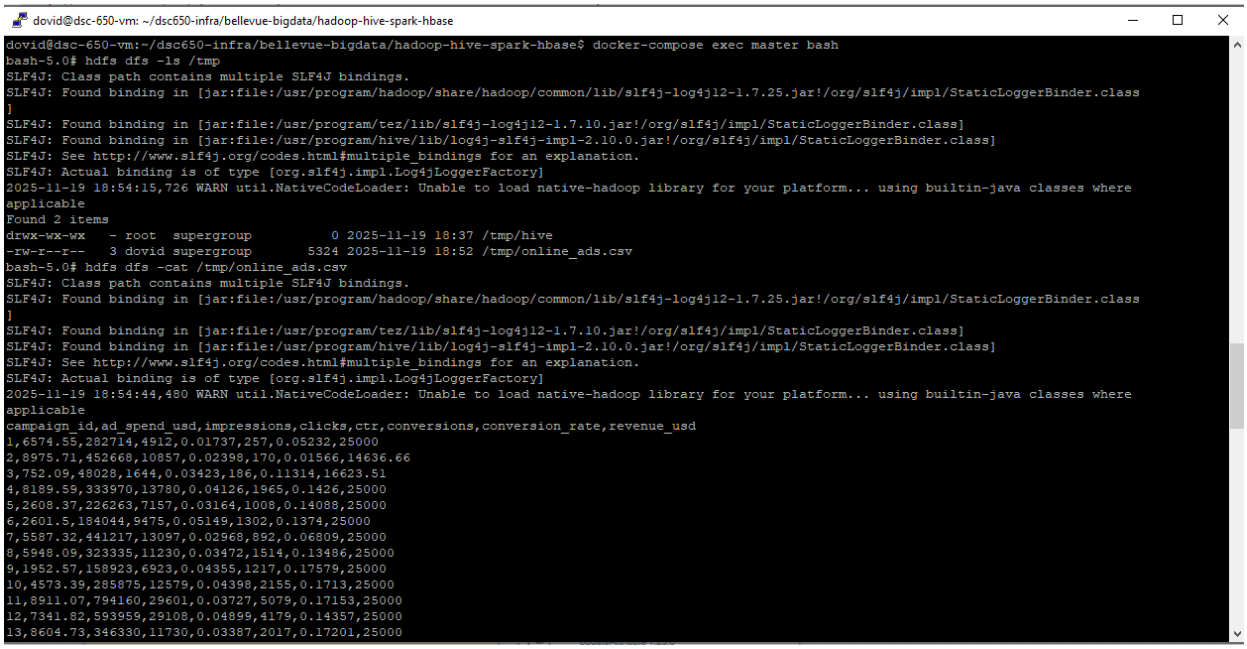

Figure 2: Terminal output confirming the presence and validity of my dataset as stored in HDFS.

In the next step, I defined a new table in Hive called “ads_raw” with corresponding columns that conform to the dataset’s types. I then loaded the CSV data into the table and selected all rows to preview and ensure that the data had been loaded correctly:

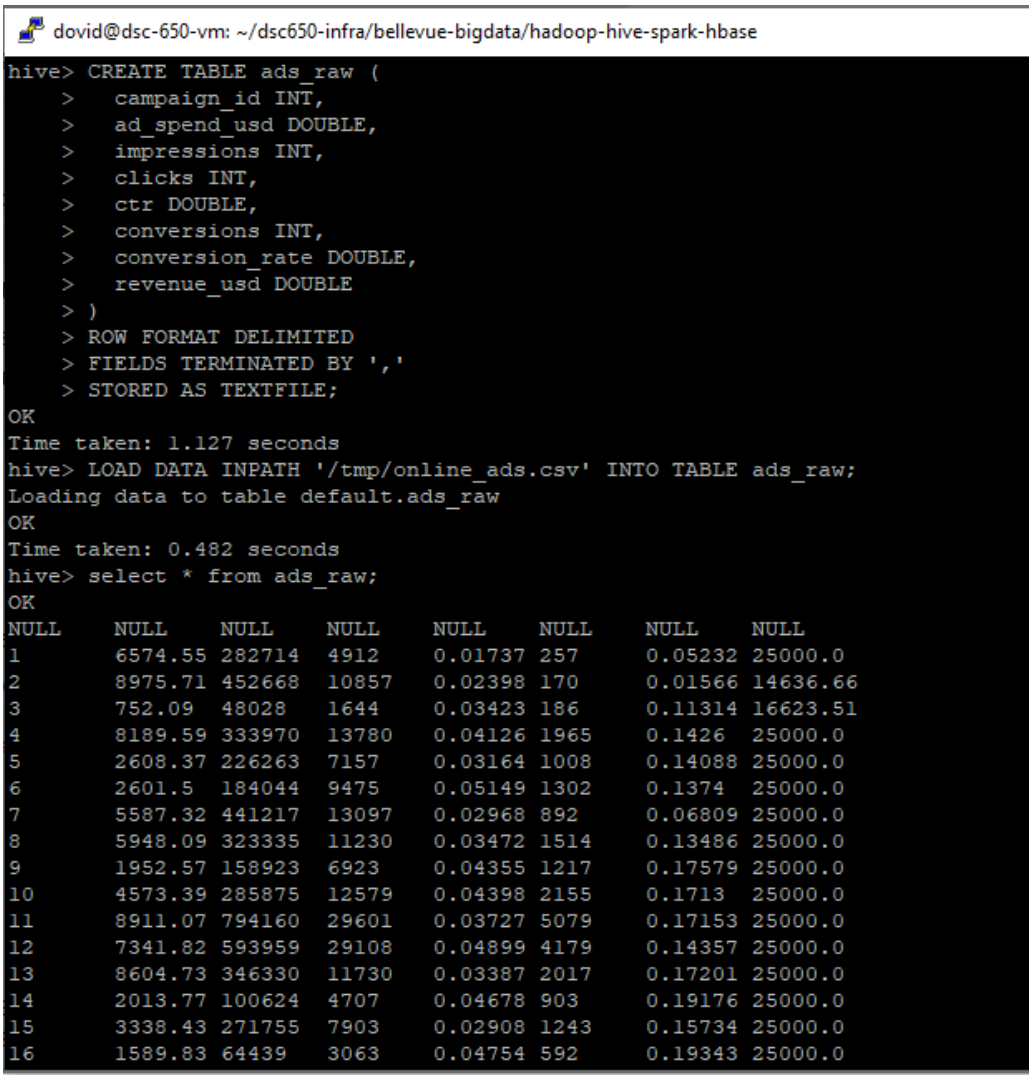

Figure 3: Hive SQL commands showing table definition, data loading, and populated “ads_raw” table rows.

As shown in Figure 3, I accidentally created a NULL row, since I didn’t explicitly tell Hive to skip the CSV header row when loading the data. Thus, I decided to run an “INSERT OVERWRITE” statement to correct my error and ensure only valid data exists in the table:

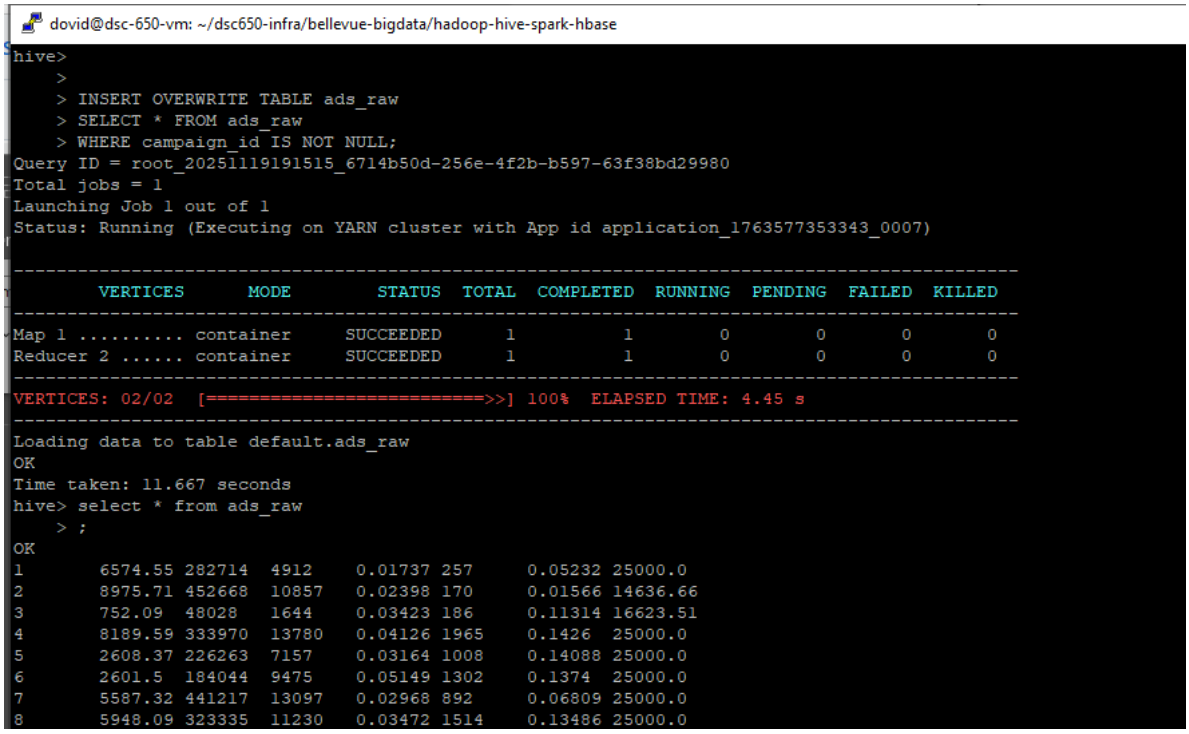

Figure 4: Deleting the accidental null row that was created from the CSV's header.

I chose INT for campaign_id because it is an incremental ID integer, DOUBLE for ad_spend_usd because it stores floating-point/decimal amounts in USD, INT for impressions because it stores whole-number counts of advertisement impressions, INT for clicks (same reason as impressions), DOUBLE for ctr (Click-Through Rate) because it is a rate expressed as a decimal, INT for conversions because it is a whole-number count of ad conversions, DOUBLE for conversion_rate since it is a rate expressed as a decimal, and DOUBLE for revenue_usd (same reason as ad_spend_usd).

I then exited out of Hive and entered HBase shell. There, I create a table and column family for my result data called “ads_metrics” and “cf”, respectively:

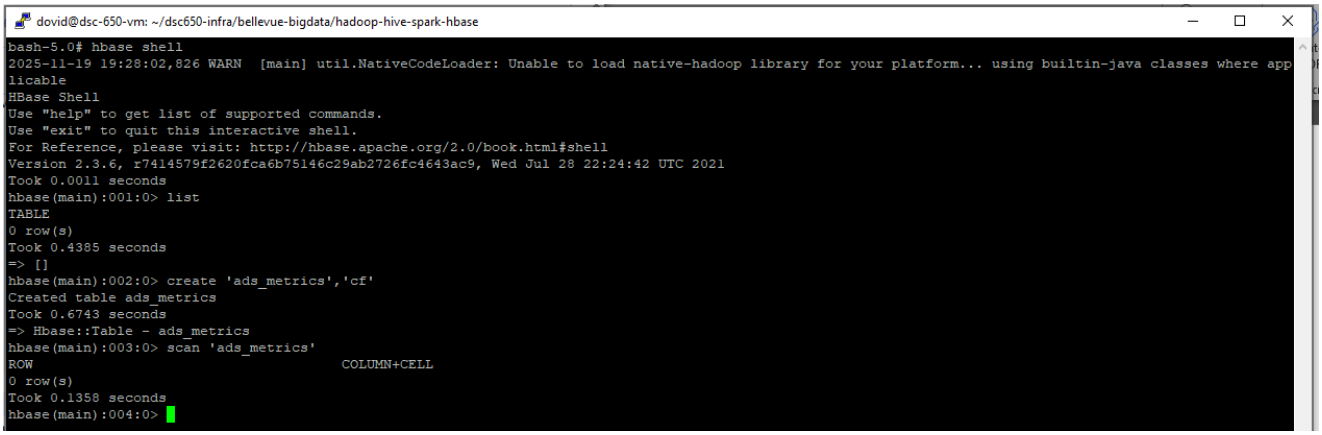

Figure 5: HBase commands to create my table and column family with confirmation.

Next, I went ahead and installed the required libraries for machine learning and data processing in Python. I installed these dependencies in both the master, worker1, and worker2 containers:

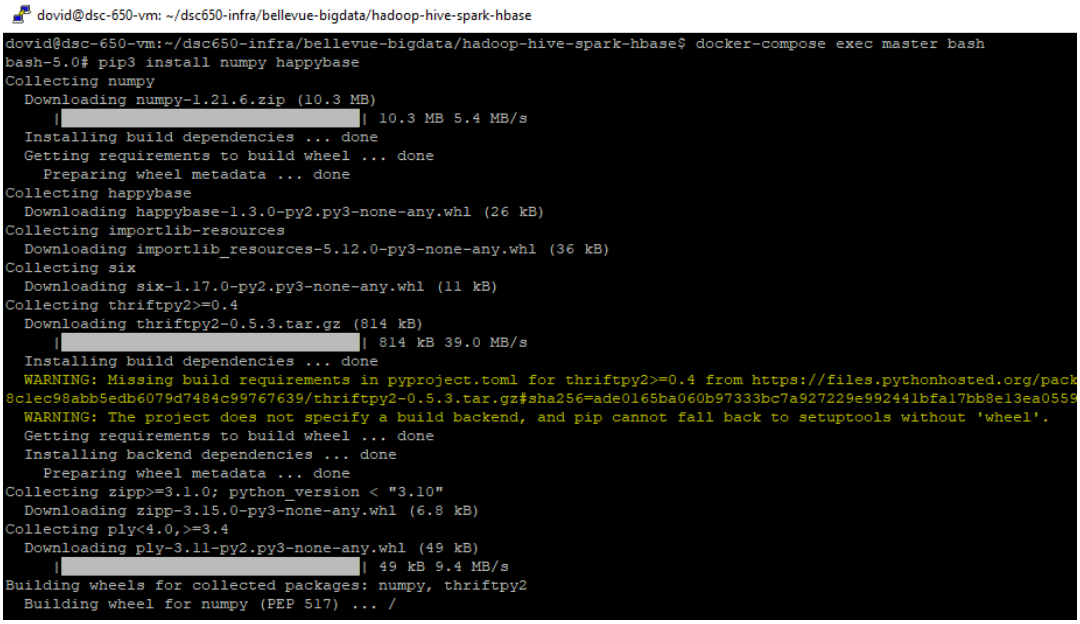

Figure 6: Installation of the required Python dependencies inside a Docker container.

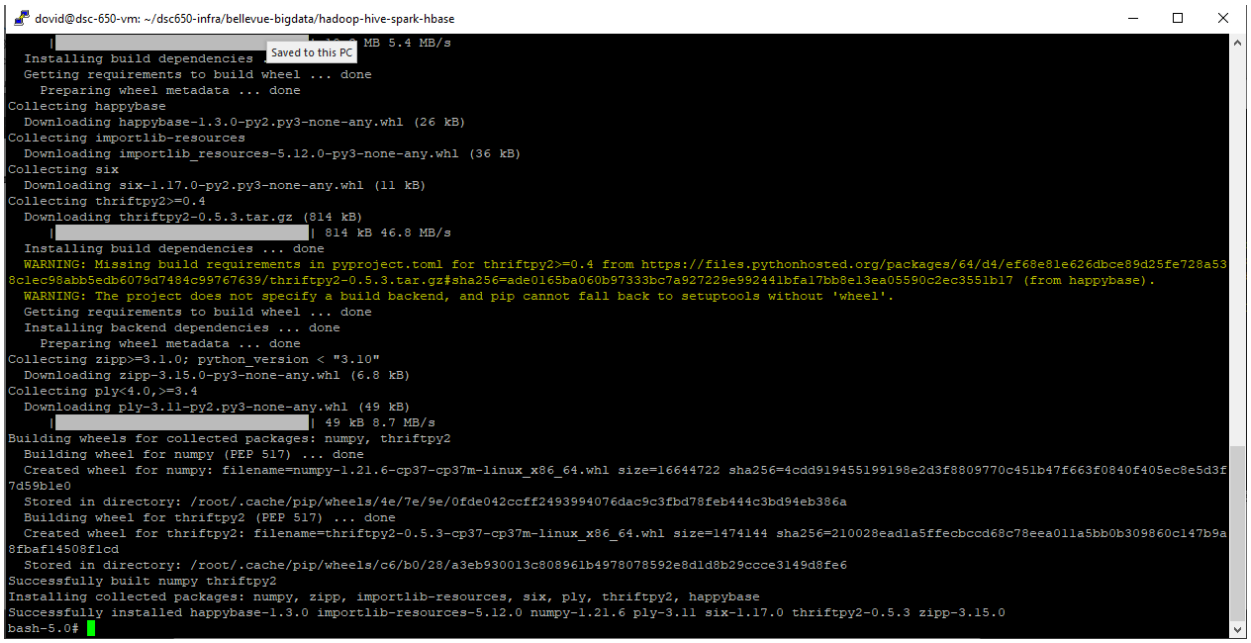

Figure 7: Output confirming successful installation of the Python dependencies for Spark. This was repeated for the master, worker1, and worker2 containers.

Next, I went into the master container and started the Thrift server, using the “nohup … &” syntax to ensure it continues running in the background, even after I exit the shell. This server creates a communication bridge between Spark and HBase. Specifically, the Thrift server listens for external events/requests and writes them to HBase.

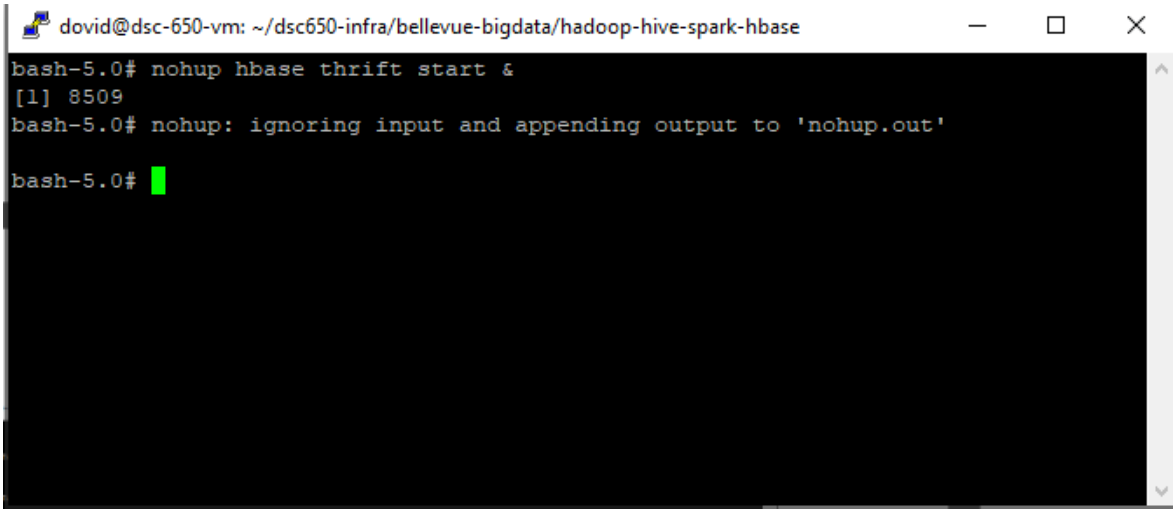

Figure 8: Starting the Thrift server in the master container's background.

I then moved on to the Spark ML steps and edited the linear regression Spark code for my dataset. Then, I pushed the code to my public DSC-650 repo.

I chose linear regression because my dataset contains continuous advertising features, such as ad spend, impressions, and clicks. These variables directly influence my continuous target variable, revenue, which makes regression a strong modeling choice.

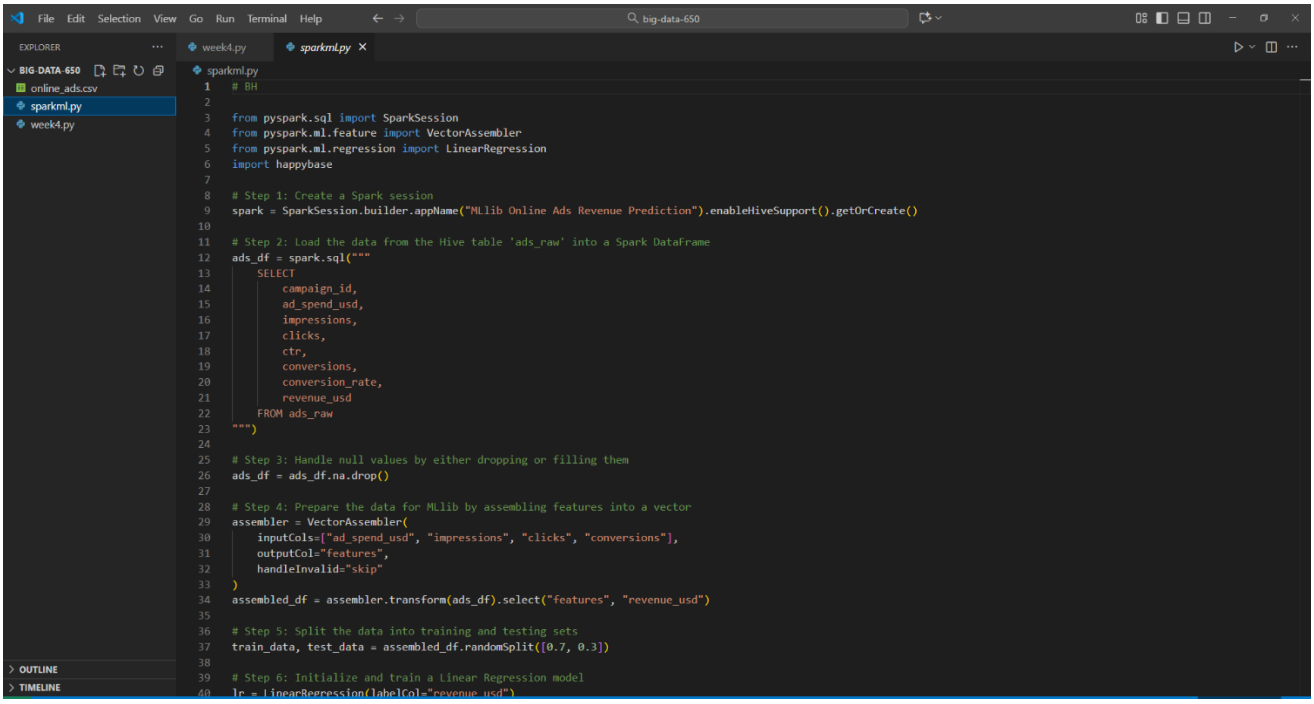

Figure 9: Editing the “sparkml.py” file in VS Code to cater it for my dataset.

Back in my VM, I cloned the repo and copied the sparkml.py script into the master container’s default directory (/usr/program).

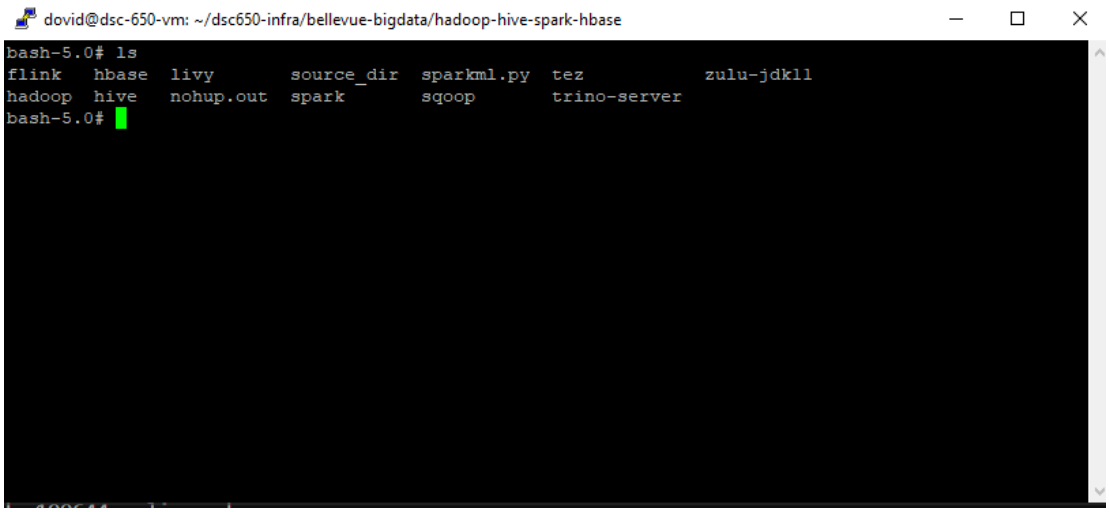

Figure 10: Confirmation that my Spark ML script was successfully copied into the VM master Docker container.

I ran the script using “spark-submit --master yarn --deploy-mode client sparkml.py”, waited for the job to finish, and opened the job’s tracking link:

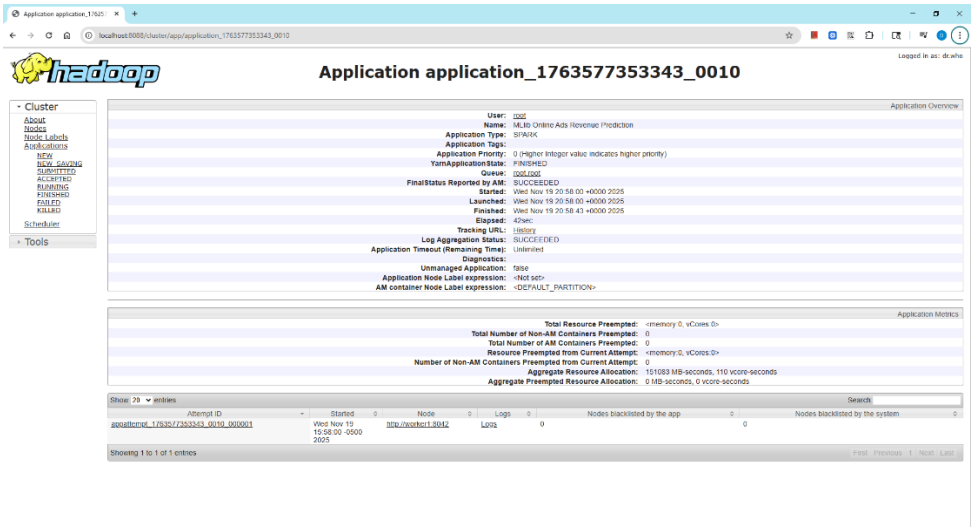

Figure 11: Hadoop UI open the job’s tracking link, showing it completed successfully.

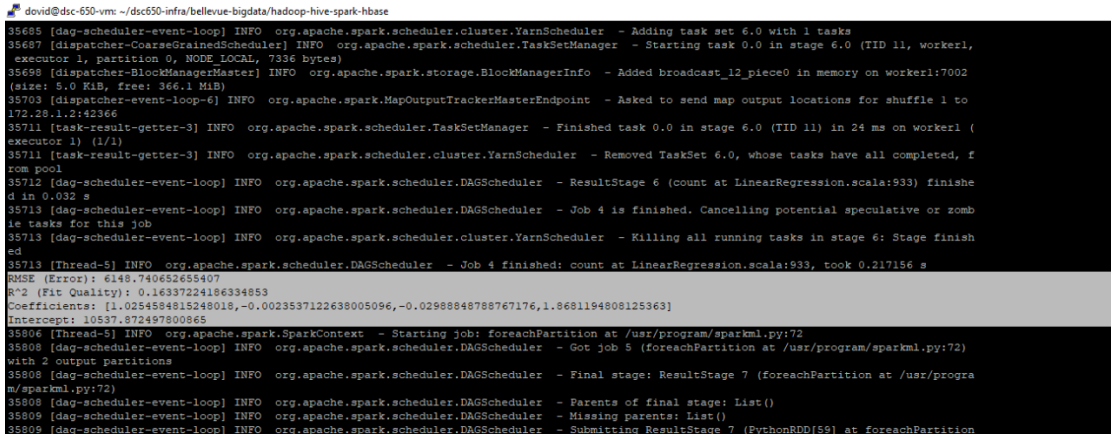

Figure 12: Printed output from job run with model evaluation metrics.

I went back into HBase and checked the job’s result (containing metrics about the Linear Regression model) by scanning the “ads_metrics” table.

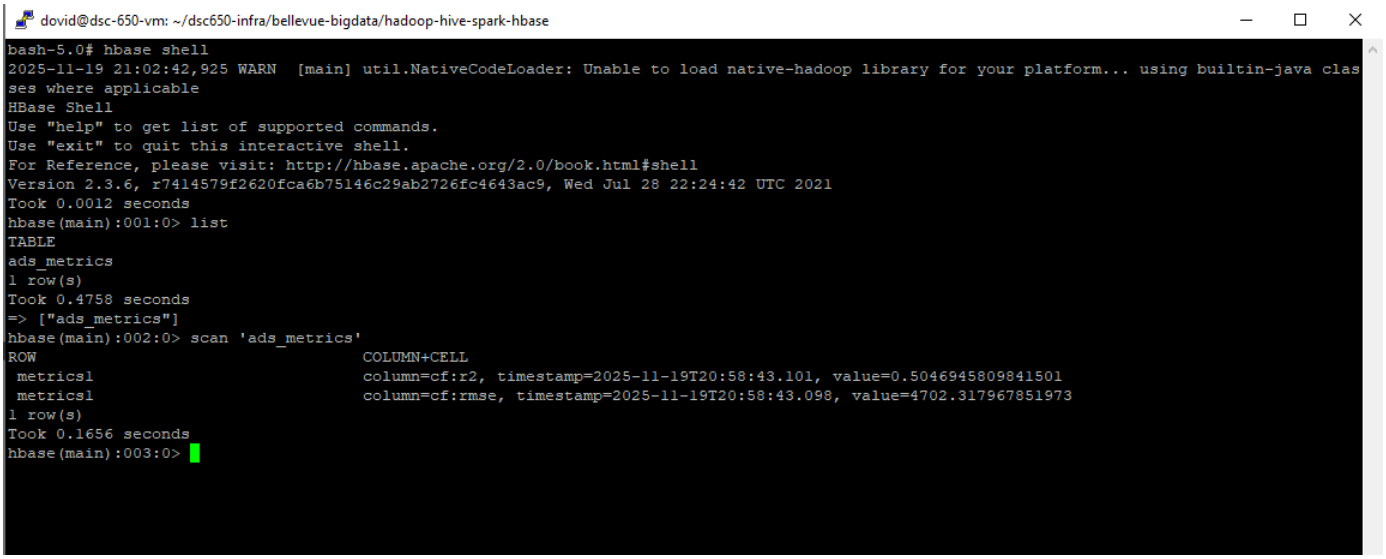

Figure 13: HBase scan output of the "ads_metrics" table, showing the stored job result from Spark ML.

Across two runs, the linear regression model produced RMSE values ranging from approximately $4,700 to $6,100 and R^2 values from 0.16 to 0.50. These figures indicate that the marketing metrics explain part of the variation in revenue, but not everything. The variance in RMSE and R^2 values across runs is most likely due to noise, different sets of data chosen randomly for the test/training splits, and the small sample size of 100 rows. My model performed moderately well all things considered, potentially reflecting realistic advertising behavior.

Code

- My Hive SQL code for defining and cleaning my table is shown above in Figures 3 and 4.
- My PySpark ML script is available here: https://github.com/dovkoy/big-data-650/blob/trunk/sparkml.py

Conclusion

In this final project, I deployed a big data pipeline to ingest online ad performance data and use it to build and evaluate a Machine Learning model. This was done by staging the dataset in Hive, training and evaluating a Linear Regression model in Spark MLib, and recording its metrics in an HBase NoSQL table. This stack covers ingestion, storage, querying, processing, and persistence using real big data technologies.

With more time, I would experiment with another algorithm, such as Random Forest or a Deep Learning Neural Network. I would also explore automating dependency management to eliminate having to manually install the same packages in each node.

In conclusion, the final project gave me experience orchestrating multiple big data services in one pipeline. This reinforces what I learned throughout the course and program, especially when it comes to truly understanding and utilizing the power of real, big data technology stacks.